



Introduction to Workflow Managers and the Persistence1 Service on Alpine

By: Maclean Sherren

What is a workflow manager?

- A workflow manager is a program that manages a collection of connected steps to fast track the computational process
- They create reproducible and scalable data analysis pipelines

Why are workflow managers important?

- Imagine having a workflow with the following steps:

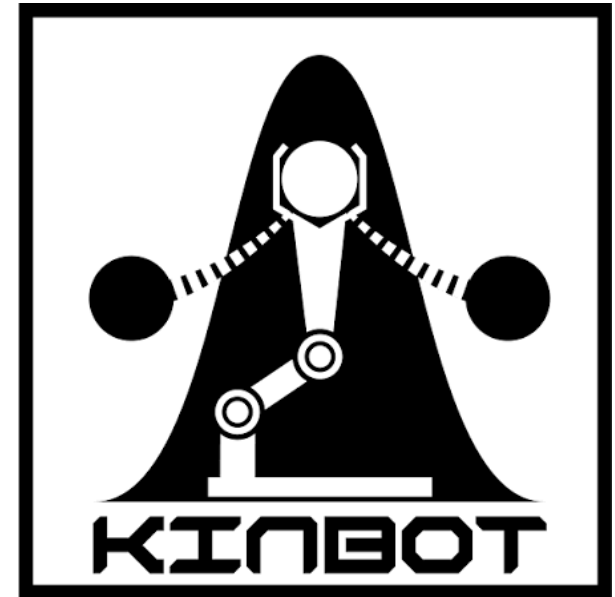
split -> data conversion/configuration -> workflow main function -> workflow outputs

Outside of a workflow manager, some of these above steps may require multiple programs, multiple jobs being run across broken up sample sizes, or a custom script to prepare the data for the main workflow function

- A workflow manager covers all these steps in a single script (Nextflow ex: <https://www.nextflow.io/basic-pipeline.html>)

Workflow Managers supported on Alpine: Nextflow, Snakemake, NF-Core, KinBot

These are the most common workflow managers used on Alpine. Nextflow, Snakemake, and NF-Core are especially utilized by Colorado Anschutz Medical Campus users



What is Persistence1 on Alpine?

- Persistence1 is a dedicated service for running long-lived workflow managers that coordinate jobs on Alpine
- It exists because of the specific demands of these managers, such as processes that remain active for days or even weeks while monitoring workflow processes and submitting jobs to the cluster
- Because of these demands, such jobs are not sustainable at scale in standard Slurm jobs and should not be run on login nodes
- Persistence1 provides a dedicated environment where users can run workflow managers and other workflow orchestration tools that require continuous execution while maintaining access to Alpine resources.

Persistence1 Disclaimer (!)

- Persistence1 is not intended for running computationally intensive workloads directly, interactive analysis, high-memory applications or replacing Slurm job submissions.
- Such analysis and **high-memory applications** should still be run through the Slurm job scheduler, though the orchestrator itself can be left running on Persistence1
- Persistence1 has a wall time of 30 days for an orchestrator to run, but the Slurm jobs submitted by the orchestrator still have a wall time of **7 days!**

For more info, see the example in the slides below

Using Persistence1 on Alpine

Users with access to the Persistence1 service can connect with:

- ssh persistence1

From there, users will have access to their /home, /projects, /scratch/alpine, and (if available) Peta Library directories

Users can:

- Run container-based workflow managers through Apptainer
- Use the Miniforge module to create and manage Conda environments that include workflow managers such as nf-core and snakemake
- Directly load and use modules for workflow managers such as Nextflow
- Persistence1 has 8 cores and 8GB of RAM available.
- The wall time is whenever the next planned maintenance is, so at the most, about a month. However; jobs submitted by orchestrators through slurm have a wall time of 7 days

Modules Available (and NOT Available) on Persistence1

Available modules:

- miniforge, nextflow versions 22.10.6, 23.04, 24.04.4, and 25.10.2, curc-quota and slurm (for slurm commands, not for running jobs!)

Notable unavailable modules:

- Nf-core, snakemake, R, python (all can be installed within a miniforge environment), as well as all other modules available on the compute side of Alpine

Nextflow Set up: Profile

- Nextflow requires considerable setup, including a config file:
 - https://github.com/kf-cuanschutz/CU-Anschutz-HPC-documentation/blob/main/Workshops/curc_alpine.config

Basic upload instructions:

- “cd /projects/\$USER/software”
- “mkdir nextflow_config”
- Upload curc_alpine.config file found in the github link above

Nextflow: Configuration options within the curc_alpine.config file

Rename “profile contact” to your name

In the process section:

- This sets the orchestrator up to use SLURM when running a series of jobs. Note here it selects all the appropriate slurm flags and gives each process a 24 hour wall time.

In the params section:

- This section is for the TOTAL orchestrator resources
- Max memory should be adjusted according to your needs
- Max cpus should be adjusted according to your needs
- Max time can be adjusted up to 7 days; this is the wall time for the total jobs run on Slurm via the nextflow orchestrator

```
/*
 * -----
 * Nextflow nf-core config file for the University of Colorado
 * Research Computing alpine HPC cluster
 * -----
 * Defines SLURM process executor and singularity settings.
 *
 */
params {
    config_profile_description = 'CURC alpine cluster profile'
    config_profile_contact = 'Username Here'
}

singularity {
    enabled = true
    cacheDir = '/projects/$USER/container_cache'
    autoMounts = true
    // /tmp is often too small -> redirect to scratch:
    runOptions = "-B /scratch/alpine/$USER:/tmp"
}

process{
    executor = 'slurm'
    queue = 'acpu'
    errorStrategy = { task.attempt <= 3 ? "retry" : "finish" }
    process.maxRetries = 3
    clusterOptions = '--account=amc-general --qos=cpu-normal'
    withLabel:process_long {
        clusterOptions = '--qos=cpu-normal'
        max_time = 24.h
    }
}

executor{
    name = 'slurm'
    queueSize = 200
    perJobMemLimit = true
    submitRateLimit = '200 / 60 min'
    killBatchSize = 50
}

params {
    max_memory = 240.GB
    max_cpus = 48
    max_time = 24.h
}

env {
    SINGULARITY_TMPDIR = "/scratch/alpine/$USER/singularity/tmp"
    SINGULARITY_CACHEDIR = "/scratch/alpine/$USER/singularity/cache"
}
```

Nextflow: Configuration options within the curc_alpine.config file

In the executor section:

- The max queue size is 200 jobs on Alpine at a time
- The submit rate limit can be adjusted depending on expected wall time for your processes; a good maximum would be 200 / 60 min
- Nextflow uses a container environment to run. The default options should be satisfactory, unless using a customized container

```
/*
 * -----
 * Nextflow nf-core config file for the University of Colorado
 * Research Computing alpine HPC cluster
 * -----
 * Defines SLURM process executor and singularity settings.
 *
 */
params {
    config_profile_description = 'CURC alpine cluster profile'
    config_profile_contact = 'Username Here'
}

singularity {
    enabled = true
    cacheDir = '/projects/$USER/container_cache'
    autoMounts = true
    // /tmp is often too small -> redirect to scratch:
    runOptions = "-B /scratch/alpine/$USER:/tmp"
}

process{
    executor = 'slurm'
    queue = 'acpu'
    errorStrategy = { task.attempt <= 3 ? "retry" : "finish" }
    process.maxRetries = 3
    clusterOptions = '--account=amc-general --qos=cpu-normal'
    withLabel:process_long {
        clusterOptions = '--qos=cpu-normal'
        max_time = 24.h
    }
}

executor{
    name = 'slurm'
    queueSize = 200
    perJobMemLimit = true
    submitRateLimit = '200 / 60 min'
    killBatchSize = 50
}

params {
    max_memory = 240.GB
    max_cpus = 48
    max_time = 24.h
}

env {
    SINGULARITY_TMPDIR = "/scratch/alpine/$USER/singularity/tmp"
    SINGULARITY_CACHEDIR = "/scratch/alpine/$USER/singularity/cache"
}
```

Nextflow pipeline: 3 options to run

There are **two effective ways** to run a Nextflow pipeline on Alpine:

- Run from persistence1, a service set up specifically for workflow managers like Nextflow. This is the recommended method
- Run from a Slurm script with the sbatch command, launched from a login node. Because of the limitations on compute, such as wall time, this method is not recommended for long jobs
 - Note: if running on slurm, you will need to create a slurm script:
<https://curc.readthedocs.io/en/latest/running-jobs/batch-jobs.html>

A Quick Slide About Tmux

- How to close a tmux session but leave it running:
“Ctrl + b”, followed by “d”. This “detaches” the session
- How to check active tmux sessions:
“tmux ls”
- How to enter a running tmux session:
“tmux attach -t session_name”
- How to cancel a tmux session:
“tmux kill-session -t session_name”
- How to kill a session you are inside:
“Ctrl + b” followed by “kill-session”

Nextflow: Persistence1 example

First, you will need to get onto the Persistence1 hardware. If you do not have access, you can request access by emailing our team at hpcsupport@cuanschultz.edu

```
# OPTIONAL: tmux; for long running workflow sessions you can come back to:  
tmux new -s my_session # -s flag names your session to come back to  
  
ssh persistence1
```

The following commands can be run from a **BASH script** (ex.: https://github.com/kf-cuanschultz/CU-Anschutz-HPC-documentation/blob/main/Workshops/bash_nextflow_examplescript.sh)
OR from a **terminal window**:

```
module load nextflow  
cd /projects/$USER/software/nextflow_config  
# This is the recommended location to store your config files
```

Notes: tmux is for longer, detached sessions when you run your actual workflow. Tmux sessions can run for weeks. Containers (apptainer, singularity) work on persistence1 with default version 1.4.3-1

Nextflow: Persistence¹ example

```
#There are export commands you can include in your bash script or simply embed in your config file. For
example:
export $pipelinepath = /path/to/your/pipeline

export $samplefile = /path/to/input/sample/file

# Once you have finished exporting any paths and commands, you are ready for your nextflow command:
nextflow run bash_run_sarek.sh -ansi-log false --input "$samplefile"--outdir
/projects/$USER/software/nextflow_config --tools mutect2,strelka,tiddit,freebayes,vep,snpeff -resume -
c curc_alpine.config
```

Note your nextflow command will BE DIFFERENT from the example above.

You will need to configure your own paths, container paths, tools, inputs, outputs, etc. This can be done either within the nextflow command, or in the config file

The last flag passes the config file, “curc_alpine.config”, which we demo-ed in previous slides. You will also need to customize your config file to fit your resource needs

Nextflow: persistence1 example script

- This example script can be found here:
https://github.com/kf-cuanschutz/CU-Anschutz-HPC-documentation/blob/main/Workshops/bash_nextflow_examplescript.sh
- When on Persistence1, this script can be run after adjustments are made. To run this outside of persistence1 from a login node, additional #SBATCH commands are required to run:
<https://curc.readthedocs.io/en/latest/running-jobs/batch-jobs.html>
- You will need to adjust the “nextflow run” command extensively to match your pipeline’s specific needs

```
1  #!/bin/bash
2  set -euo pipefail
3
4  # make software accessible:
5  module load nextflow
6
7  echo _____
8  nextflow -version
9  echo _____
10
11
12  ## Override defaults set by nextflow module:
13  export NXF_WORK="$scrproj/work"
14  export NXF_TEMP="$scrproj/tmp"
15  export NXF_HOME="/projects/$USER/software/nextflow_config"
16  export _JAVA_OPTIONS="-Xmx16G" # increase heap to avoid java.lang.OutOfMemoryError
17
18
19
20  nextflow run "$pipelinepath" -ansi-log false \
21  --input "$samplefile" --trim_fastq \
22  --save_trimmed --save_mapped --save_output_as_bam --outdir "$pipeoutdir" \
23  --tools mutect2,strelda,tiddit,freebayes,vep,snpEff -resume \
24  -c curc_alpine.config,fastp.config
```

Nf-core set up on persistence1



Tmux is optional to start

```
ssh persistence1  
module load miniforge  
conda create --name nf-core-env  
conda activate nf-core_env  
conda install bioconda::nf-core  
nf-core interface
```

The final command launches the nf-core interface, where you can install modules, pipelines, or run test datasets.

Note that the nf-core install also installs nextflow, so to run a nf-core pipeline, a command such as “nextflow run nf-core \ <nextflow pipeline flags here>” will work.

Once nf-core is installed, you can also launch a script within persistence1 (see slides12-14)

Snakemake installation on persistence1

*Tmux session is always a possibility before “ssh persistence1”



```
ssh persistence1  
  
module load miniforge  
conda create -n snakemake_env  
  
conda activate snakemake_env  
  
conda install -c bioconda snakemake  
  
#Test  
  
snakemake --help
```

Snakemake basic example on persistence1:

Snakemake command run with 1 core

Input file is “snakefile”

Output file “hello_world”

The following can also be added to a bash script, requesting compute resources, and run in a tmux window:

```
module load miniforge
conda activate snakemake_env
snakemake --cores 4
```

```
(snakemake_env) [msherren@xsede.org@persistence1 snakemake]$ snakemake --cores 1 -n
Assuming unrestricted shared filesystem usage.

SNAKEMAKE
=====
Date: 2026-10-06 11:26:09
Workflow ID: 420c466d-5b33-4e00-87c7-ef3eff05998f
Platform: Linux-4.18.0-553.124.1.el8_10.x86_64-x86_64-with-glibc2.28
Host: persistence1
User: msherren@xsede.org
Snakemake version: 9.27.0
Python version: 3.14.8 | packaged by conda-forge | (main, Oct 6 2026, 13:38:01) [GCC 15.3.0]
Command: /projects/msherren@xsede.org/software/anaconda/envs/snakemake_env/bin/snakemake --cores 1 -n
Snakefile: /projects/msherren@xsede.org/software/snakemake/snakefile
Base directory: /projects/msherren@xsede.org/software/snakemake
Run directory: /projects/msherren@xsede.org/software/snakemake
Working directory: /projects/msherren@xsede.org/software/snakemake
Config file(s): []
Config MD5: 99914b932bd37a50b983c5e7c90ae93b

Building DAG of jobs...
Job stats:
job          count
-----
hello_world    1
total          1
```

This will ultimately run the snakemake workflow out of your current working directory with 4 cores on persistence1

Troubleshooting on persistence1

- Persistence1 acts in some ways as its own login node. When a pipeline is launched from persistence1, it utilizes slurm resources and generates JobIDs from each individual process.
- You can then troubleshoot the jobs in a traditional manner with commands such as “seff”, “squeue”, “sacct”. You can read more about troubleshooting jobs on Alpine here:

https://github.com/kf-cuanschultz/CU-Anschutz-HPC-documentation/blob/main/Workshops/Alpine_best_practices_workshop_03262025.pdf.

- Another way to troubleshoot is with the output directory. In the nextflow example, the output directory was set as “/projects/\$USER/software/nextflow_config”
 - When a nextflow pipeline fails, a “.nextflow.log.[#]” file is generated in the output directory. You can see these hidden files with the command “ls -la”:

```
[msherren@xsede.org@persistence1 ~]$ singularity --version
apptainer version 1.4.3-1
[msherren@xsede.org@persistence1 ~]$ ssh persistence1
Welcome to CU-Boulder Research Computing.

* Website http://colorado.edu/rc
* Questions? rc-help@colorado.edu
* Subscribe to system announcements: https://curc.statuspage.io/
* Please type rc-help for the Acceptable Use Policy and a short help page.

You are using login node: persistence1

Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-dashboard
Last login: Tue Oct  6 10:00:32 2026 from 10.225.160.190
Welcome to CU-Boulder Research Computing.
```

```
[msherren@xsede.org@persistence1 nextflow_config]$ ls -la
total 592
drwxr-sr-x  5 msherren@xsede.org msherrengrp 373 Oct  6 10:11 .
drwxr-sr-x 92 msherren@xsede.org msherrengrp 3385 Oct  5 09:32 ..
-rwxrwxr-x  1 msherren@xsede.org msherrengrp 801 Oct  5 09:33 bash_run_sarek.sh
-rw-r--r--  1 msherren@xsede.org msherrengrp 1247 Oct  5 19:57 curc_alpine.config
drwxrwsr-x  4 msherren@xsede.org msherrengrp  69 Oct  6 10:11 .nextflow
-rw-rw-r--  1 msherren@xsede.org msherrengrp 6833 Oct  6 10:11 .nextflow.log
-rw-rw-r--  1 msherren@xsede.org msherrengrp 2131 Oct  6 10:11 .nextflow.log.1
-rw-rw-r--  1 msherren@xsede.org msherrengrp 5515 Oct  5 19:57 .nextflow.log.2
-rw-rw-r--  1 msherren@xsede.org msherrengrp 5488 Oct  5 19:56 .nextflow.log.3
-rw-rw-r--  1 msherren@xsede.org msherrengrp  372 Oct  5 19:56 .nextflow.log.4
-rw-rw-r--  1 msherren@xsede.org msherrengrp  566 Oct  5 19:49 .nextflow.log.5
-rw-rw-r--  1 msherren@xsede.org msherrengrp  389 Oct  5 10:46 .nextflow.log.6
drwxrwsr-x  2 msherren@xsede.org msherrengrp   0 Oct  5 19:56 plugins
drwxrwsr-x  4 msherren@xsede.org msherrengrp  47 Oct  5 10:46 tmp
```



Questions?

THANK YOU

Feel free to contact us at
HPCSupport@cuanschutzh.edu for access to
persistence1 and any questions related to
workflow managers on persistence1

